

```

In [16]: import os
import json
import csv
import matplotlib.pyplot as plt

# --- 1. DIRECTORY CONFIGURATION ---
DIRS = {
    "ADA": r"C:\Users\vishw\OneDrive\Desktop\Explo\stylegan2-ADA-pytorch\training-r",
    "FakeCLR": r"C:\Users\vishw\OneDrive\Desktop\Explo\FakeCLR\merged_snapshots",
    "InsGen": r"C:\Users\vishw\OneDrive\Desktop\Explo\insgen\training-runs\00003-pa",
    "DANI": r"C:\Users\vishw\OneDrive\Desktop\Explo\DANI\training-runs\00004-styleg
}

COLORS = {"ADA": "#1f77b4", "FakeCLR": "#d62728", "InsGen": "#2ca02c", "DANI": "#94
plt.style.use('seaborn-v0_8-whitegrid')

# --- 2. PARSING FUNCTIONS ---
def load_metric(base_dir, metric_type):
    variants = {
        'kid': [('kid5k_full', ['kid5k_full']), ('kid50k_full', ['kid50k_full'])],
        'fid': [('fid5k_full', ['fid5k_full']), ('fid50k_full', ['fid50k_full'])],
        'pr': [('pr5k3_full', ['pr5k3_full_precision', 'pr5k3_full_recall']),
                ('pr50k3_full', ['pr50k3_full_precision', 'pr50k3_full_recall'])],
        'is': [('is5k', ['is5k_mean']), ('is50k', ['is50k_mean'])]
    }

    data = {}
    for file_name, keys in variants[metric_type]:
        file_path = os.path.join(base_dir, f"metric-{file_name}.jsonl")
        if os.path.exists(file_path):
            with open(file_path, 'rt') as f:
                for line in f:
                    entry = json.loads(line)
                    pkl_name = os.path.basename(entry['snapshot_pkl'])
                    kimg = int(pkl_name.split('-')[-1].split('.')[0])

                    if metric_type == 'kid': data[kimg] = {'score': entry['results']}
                    if metric_type == 'fid': data[kimg] = {'score': entry['results']}
                    if metric_type == 'pr': data[kimg] = {'precision': entry['results']}
                    if metric_type == 'is': data[kimg] = {'score': entry['results']}

    return data

def load_custom_csv(file_path, value_col):
    data = {}
    if os.path.exists(file_path):
        with open(file_path, 'r') as f:
            reader = csv.DictReader(f)
            for row in reader:
                data[int(row['kimg'])] = {'score': float(row[value_col])}
    return data

def extract_xy(data_dict, key):
    sorted_kimgs = sorted(data_dict.keys())

```

```

    return sorted_kings, [data_dict[k][key] for k in sorted_kings]

# --- 3. LOAD ALL DATA ---
data_store = {}
for name, path in DIRS.items():
    data_store[name] = {
        'kid': load_metric(path, 'kid'),
        'fid': load_metric(path, 'fid'),
        'pr': load_metric(path, 'pr'),
        'is': load_metric(path, 'is'),
        'lpips': load_custom_csv(os.path.join(path, "custom_lpips_metrics.csv"), "l
        'msssim': load_custom_csv(os.path.join(path, "custom_msssim_metrics.csv"),
    }
print("data successfully loaded")

```

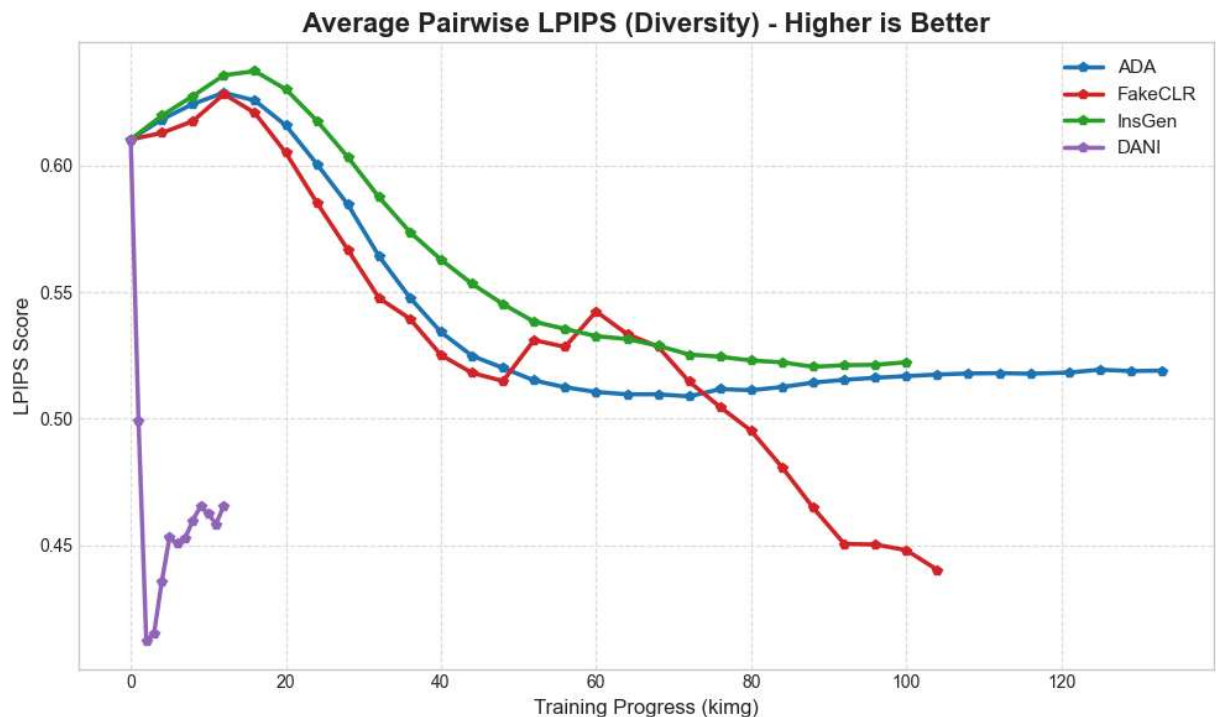
data successfully loaded

```

In [17]: plt.figure(figsize=(10, 6))
for model_name, metrics in data_store.items():
    if not metrics['lpips']: continue
    x, y = extract_xy(metrics['lpips'], 'score')
    plt.plot(x, y, marker='p', linewidth=2.5, color=COLORS[model_name], label=model

plt.title('Average Pairwise LPIPS (Diversity) - Higher is Better', fontsize=16, fon
plt.xlabel('Training Progress (king)', fontsize=12)
plt.ylabel('LPIPS Score', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend(fontsize=11)
plt.tight_layout()
plt.show()

```



```

In [18]: plt.figure(figsize=(10, 6))
for model_name, metrics in data_store.items():
    if not metrics['kid']: continue

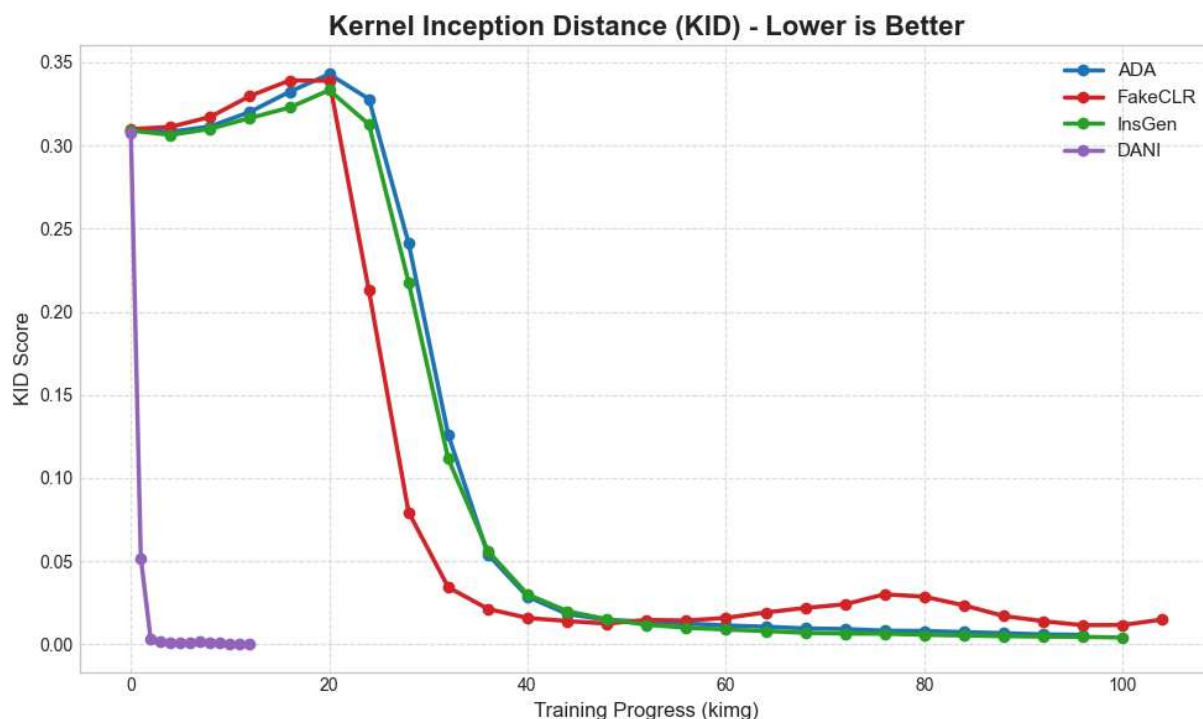
```

```

x, y = extract_xy(metrics['kid'], 'score')
plt.plot(x, y, marker='o', linewidth=2.5, color=COLORS[model_name], label=model

plt.title('Kernel Inception Distance (KID) - Lower is Better', fontsize=16, fontwei
plt.xlabel('Training Progress (king)', fontsize=12)
plt.ylabel('KID Score', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend(fontsize=11)
plt.tight_layout()
plt.show()

```



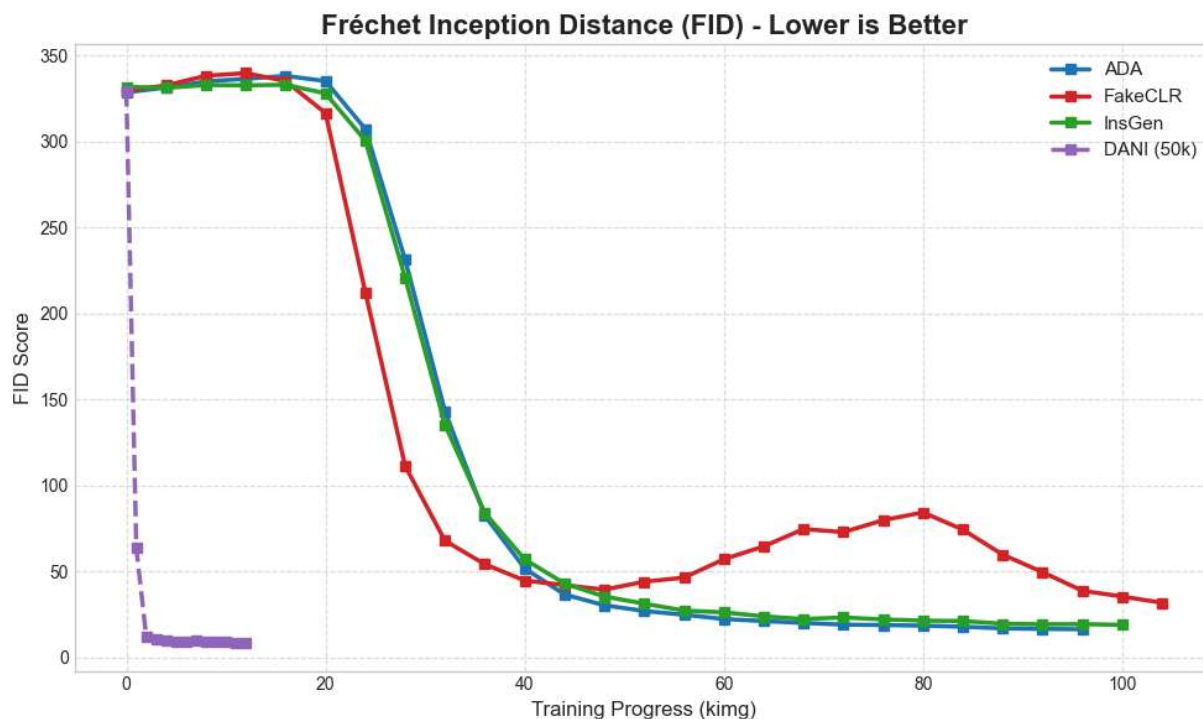
```

In [19]: plt.figure(figsize=(10, 6))
for model_name, metrics in data_store.items():
    if not metrics['fid']: continue
    x, y = extract_xy(metrics['fid'], 'score')

    # Keeping the dashed line style for DANI's 50k run
    line_style = '--' if model_name == "DANI" else '-'
    plt.plot(x, y, marker='s', linestyle=line_style, linewidth=2.5, color=COLORS[mo

plt.title('Fréchet Inception Distance (FID) - Lower is Better', fontsize=16, fontwei
plt.xlabel('Training Progress (king)', fontsize=12)
plt.ylabel('FID Score', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend(fontsize=11)
plt.tight_layout()
plt.show()

```



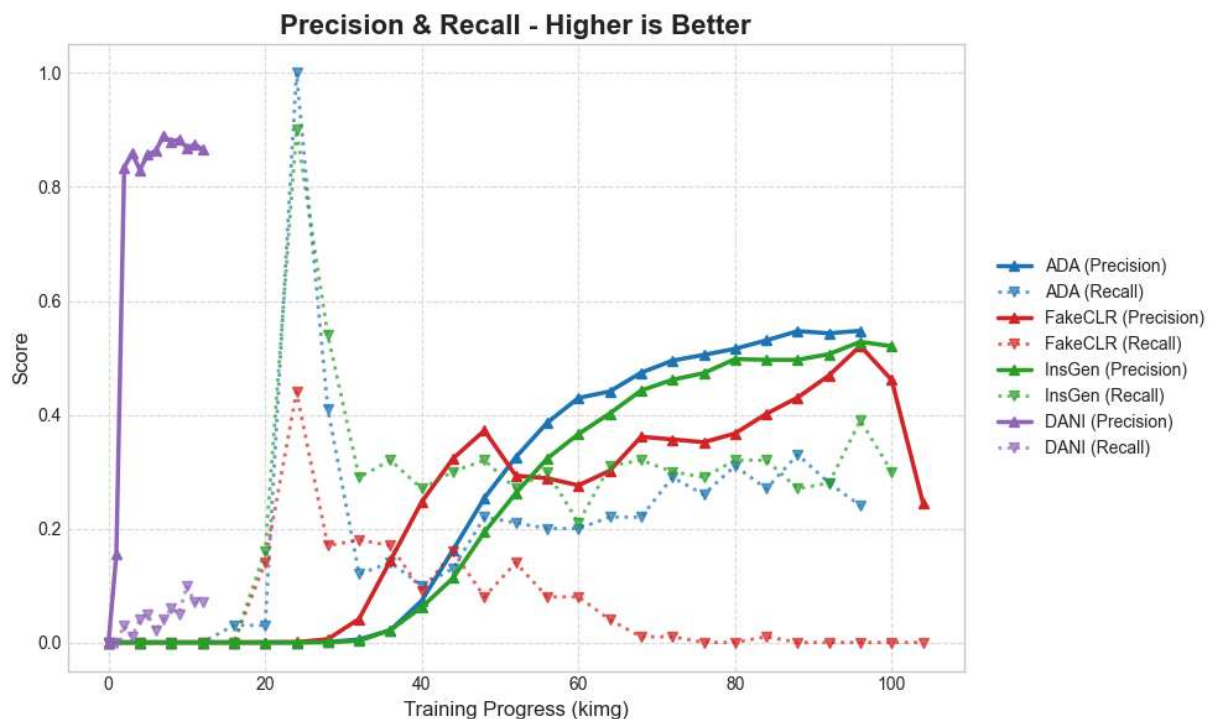
```
In [20]: plt.figure(figsize=(10, 6))
for model_name, metrics in data_store.items():
    if not metrics['pr']: continue

    # Precision (Fidelity)
    x_p, y_p = extract_xy(metrics['pr'], 'precision')
    plt.plot(x_p, y_p, marker='^', linestyle='--', linewidth=2.5, color=COLORS[model_name])

    # Recall (Diversity)
    x_r, y_r = extract_xy(metrics['pr'], 'recall')
    plt.plot(x_r, y_r, marker='v', linestyle=':', linewidth=2, alpha=0.7, color=COLORS[model_name])

plt.title('Precision & Recall - Higher is Better', fontsize=16, fontweight='bold')
plt.xlabel('Training Progress (king)', fontsize=12)
plt.ylabel('Score', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)

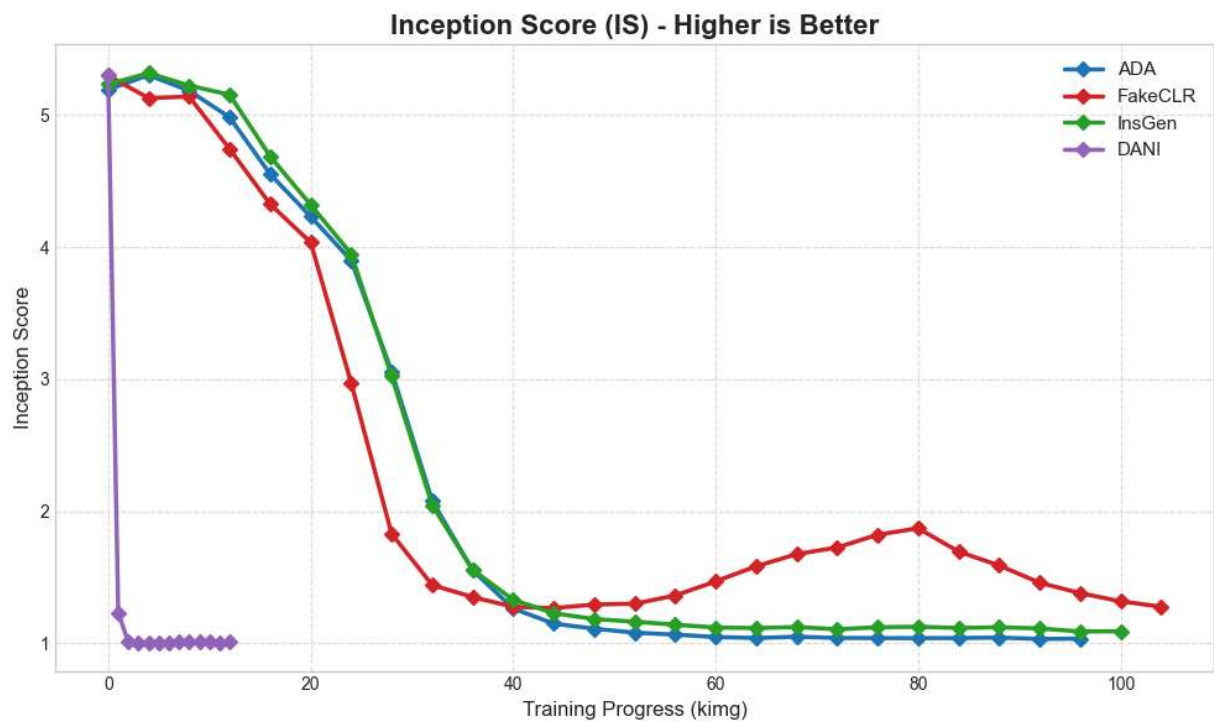
# Adjusted Legend to fit nicely on the smaller graph
plt.legend(fontsize=10, loc='center left', bbox_to_anchor=(1.02, 0.5))
plt.tight_layout()
plt.show()
```



```
In [21]: plt.figure(figsize=(10, 6))

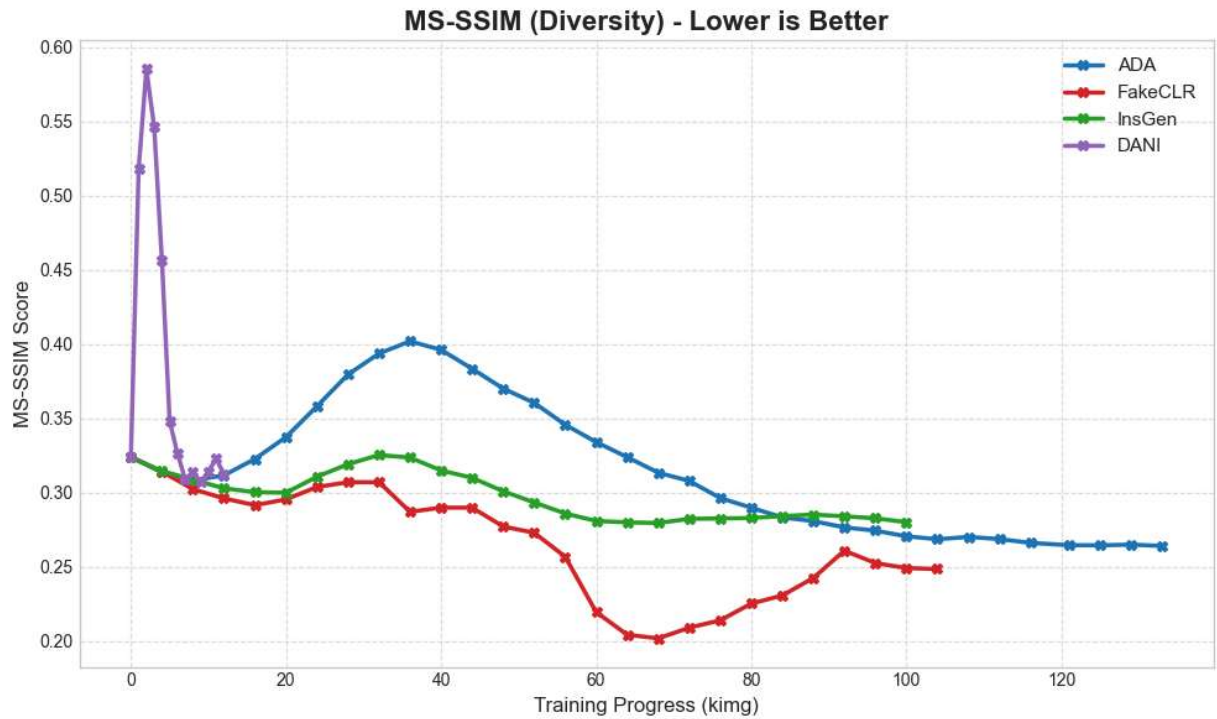
for model_name, metrics in data_store.items():
    if not metrics['is']: continue
    x, y = extract_xy(metrics['is'], 'score')
    plt.plot(x, y, marker='D', linewidth=2.5, color=COLORS[model_name], label=model_name)

plt.title('Inception Score (IS) - Higher is Better', fontsize=16, fontweight='bold')
plt.xlabel('Training Progress (kimg)', fontsize=12)
plt.ylabel('Inception Score', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend(fontsize=11)
plt.tight_layout()
plt.show()
```



```
In [22]: plt.figure(figsize=(10, 6))
for model_name, metrics in data_store.items():
    if not metrics['msssim']: continue
    x, y = extract_xy(metrics['msssim'], 'score')
    plt.plot(x, y, marker='X', linewidth=2.5, color=COLORS[model_name], label=model_name)

plt.title('MS-SSIM (Diversity) - Lower is Better', fontsize=16, fontweight='bold')
plt.xlabel('Training Progress (king)', fontsize=12)
plt.ylabel('MS-SSIM Score', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend(fontsize=11)
plt.tight_layout()
plt.show()
```



```
In [23]: # --- THE MASTER PLOT (2x3 GRID) ---
# (Assumes data_store, COLORS, and extract_xy are already loaded in memory)

fig, axs = plt.subplots(2, 3, figsize=(24, 14))
fig.suptitle("Master Architecture Comparison: FFHQ to Panda Transfer Learning", font

# Helper for Line styles (Dashed for DANI's 50k run to distinguish it)
def get_style(model_name): return '--' if model_name == "DANI" else '-'
def get_label(model_name): return f"{model_name} (50k)" if model_name == "DANI" else

# [0,0] FID
for model, metrics in data_store.items():
    if not metrics['fid']: continue
    x, y = extract_xy(metrics['fid'], 'score')
    axs[0,0].plot(x, y, marker='s', linestyle=get_style(model), linewidth=2.5, color=
axs[0,0].set_title('Fréchet Inception Distance (FID) ↓', fontsize=16, fontweight='b
axs[0,0].set_ylabel('Score', fontsize=12)
axs[0,0].grid(True, linestyle='--', alpha=0.6)
axs[0,0].legend(fontsize=11)

# [0,1] KID
for model, metrics in data_store.items():
    if not metrics['kid']: continue
    x, y = extract_xy(metrics['kid'], 'score')
    axs[0,1].plot(x, y, marker='o', linestyle=get_style(model), linewidth=2.5, color=
axs[0,1].set_title('Kernel Inception Distance (KID) ↓', fontsize=16, fontweight='b
axs[0,1].grid(True, linestyle='--', alpha=0.6)
axs[0,1].legend(fontsize=11)

# [0,2] LPIPS
for model, metrics in data_store.items():
    if not metrics['lpips']: continue
    x, y = extract_xy(metrics['lpips'], 'score')
```

```

    axs[0,2].plot(x, y, marker='p', linewidth=2.5, color=COLORS[model], label=model)
axs[0,2].set_title('Pairwise LPIPS (Diversity) ↑', fontsize=16, fontweight='bold')
axs[0,2].grid(True, linestyle='--', alpha=0.6)
axs[0,2].legend(fontsize=11)

# [1,0] Inception Score (IS)
for model, metrics in data_store.items():
    if not metrics['is']: continue
    x, y = extract_xy(metrics['is'], 'score')
    axs[1,0].plot(x, y, marker='D', linestyle=get_style(model), linewidth=2.5, color=COLORS[model])
axs[1,0].set_title('Inception Score (IS) ↑', fontsize=16, fontweight='bold')
axs[1,0].set_xlabel('Training Progress (kimg)', fontsize=12)
axs[1,0].set_ylabel('Score', fontsize=12)
axs[1,0].grid(True, linestyle='--', alpha=0.6)
axs[1,0].legend(fontsize=11)

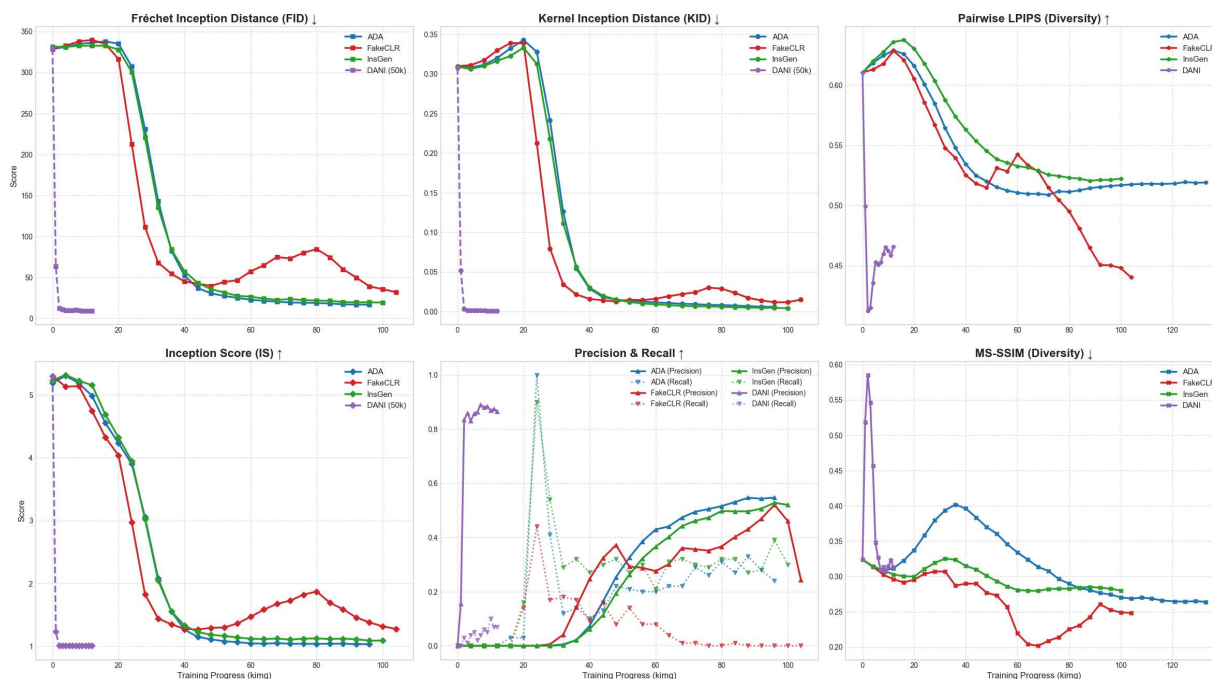
# [1,1] Precision & Recall
for model, metrics in data_store.items():
    if not metrics['pr']: continue
    x_p, y_p = extract_xy(metrics['pr'], 'precision')
    x_r, y_r = extract_xy(metrics['pr'], 'recall')
    axs[1,1].plot(x_p, y_p, marker='^', linestyle='--', linewidth=2.5, color=COLORS[model])
    axs[1,1].plot(x_r, y_r, marker='v', linestyle=':', linewidth=2, alpha=0.7, color=COLORS[model])
axs[1,1].set_title('Precision & Recall ↑', fontsize=16, fontweight='bold')
axs[1,1].set_xlabel('Training Progress (kimg)', fontsize=12)
axs[1,1].grid(True, linestyle='--', alpha=0.6)
# FIX: Moved Legend inside, using 2 columns to keep it compact
axs[1,1].legend(fontsize=10, loc='best', ncol=2)

# [1,2] MS-SSIM
for model, metrics in data_store.items():
    if not metrics['msssim']: continue
    x, y = extract_xy(metrics['msssim'], 'score')
    axs[1,2].plot(x, y, marker='X', linewidth=2.5, color=COLORS[model], label=model)
axs[1,2].set_title('MS-SSIM (Diversity) ↓', fontsize=16, fontweight='bold')
axs[1,2].set_xlabel('Training Progress (kimg)', fontsize=12)
axs[1,2].grid(True, linestyle='--', alpha=0.6)
axs[1,2].legend(fontsize=11)

# FIX: Explicit padding controls to distribute whitespace evenly
plt.tight_layout(pad=2.0, w_pad=1.5, h_pad=2.0)
plt.show()

```

Master Architecture Comparison: FFHQ to Panda Transfer Learning



```
In [24]: import pandas as pd
from IPython.display import display

# --- FINAL METRICS SUMMARY TABLE ---
# (Assumes data_store and extract_xy are already loaded in memory)

table_rows = []

for model_name, metrics in data_store.items():
    # Label DANI appropriately to remember it's the 50k run
    display_name = f"{model_name} (50k)" if model_name == "DANI" else model_name
    row_data = {"Architecture": display_name}

    # Helper function to safely extract the last recorded value
    def get_last_val(metric_data, key='score'):
        if metric_data:
            _, y = extract_xy(metric_data, key)
            return f"{y[-1]:.4f}"
        return "N/A"

    # Populate the row
    row_data["FID ↓"] = get_last_val(metrics.get('fid'))
    row_data["KID ↓"] = get_last_val(metrics.get('kid'))
    row_data["Inception Score ↑"] = get_last_val(metrics.get('is'))

    # Precision and Recall have specific keys
    if metrics.get('pr'):
        _, p_vals = extract_xy(metrics['pr'], 'precision')
        _, r_vals = extract_xy(metrics['pr'], 'recall')
        row_data["Precision (Fidelity) ↑"] = f"{p_vals[-1]:.4f}"
        row_data["Recall (Diversity) ↑"] = f"{r_vals[-1]:.4f}"
    else:
        row_data["Precision (Fidelity) ↑"] = "N/A"
```

```
row_data["Recall (Diversity) ↑"] = "N/A"

row_data["LPIPS (Diversity) ↑"] = get_last_val(metrics.get('lpips'))
row_data["MS-SSIM (Diversity) ↓"] = get_last_val(metrics.get('msssim'))

table_rows.append(row_data)

# Create and display the DataFrame
df_summary = pd.DataFrame(table_rows)
df_summary.set_index("Architecture", inplace=True)

# Apply some basic pandas styling for a cleaner look
styled_df = df_summary.style.set_caption("Final Transfer Learning Metric Comparison")
                        .set_table_styles([
                            {'selector': 'caption',
                             'props': [('font-size', '18px'), ('font-weight', 'bold')]}
                        ]) \
                        .set_properties(**{'text-align': 'center', 'font-size': '12px'})
                        .set_table_attributes('style="width: 100%; border-collapse: collapse;"')

# Render the table
display(styled_df)
```

Final Transfer Learning Metric Comparison

	FID ↓	KID ↓	Inception Score ↑	Precision (Fidelity) ↑	Recall (Diversity) ↑	LPIPS (Diversity) ↑	MS-SSIM (Diversity) ↓
Architecture							
ADA	16.4066	0.0057	1.0354	0.5474	0.2400	0.5190	0.2640
FakeCLR	31.8215	0.0149	1.2773	0.2446	0.0000	0.4403	0.2485
InsGen	18.9917	0.0041	1.0912	0.5204	0.3000	0.5223	0.2800
DANI (50k)	8.7912	0.0006	1.0079	0.8661	0.0700	0.4657	0.3125